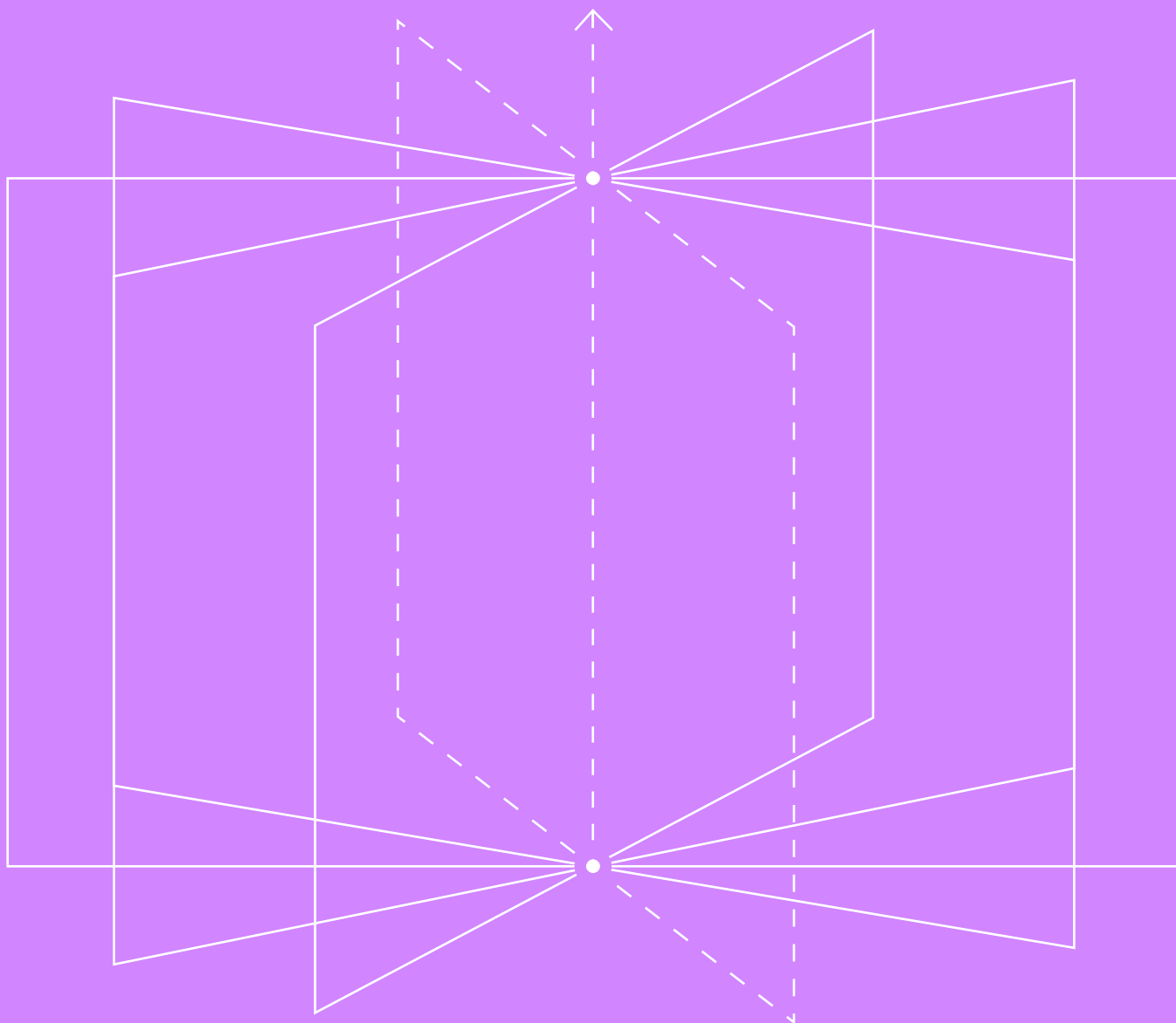


Оконные функции



SQL И БАЗЫ ДАННЫХ (ДЛЯ РАЗРАБОТЧИКОВ)

ЛОНГРИД

НЕДЕЛЯ 6

Что такое оконная функция

При работе с данными часто приходится проводить дополнительные вычисления над строками или проводить ранжирование наблюдений (то есть **проставление рангов**), не меняя при этом структуру таблицы.

ОПРЕДЕЛЕНИЯ

Оконные функции производят множество различных операций, таких как агрегации, ранжирование, текстовые операции и многие другие. Их отличительной чертой является то, что они исполняются на уровне каждой строки (как и обычные функции), но для вычисления могут использовать не только данную строку, но и другие строки, которые находятся в рамках соответствующего **окна**.

Окно — это уникальное значение в рамках того или иного столбца (или комбинации столбцов).

Например, если строка с идентификатором заказа 1111 относится к городу Москве (city = “Москва”), то проставление в качестве окна столбца city проведёт вычисление над всеми строками, относящимися к Москве, и проставит результат вычисления каждой строке, относящейся к Москве.

Принято разделять оконные функции на 3 основных вида: агрегирующие, ранжирующие и оконные функции смещения.

Агрегирующие функции

Мы уже знакомились с агрегирующими функциями, например с **COUNT**, **SUM**, **AVG**, **MIN**, **MAX**, а также с оператором группировки **GROUP BY**. Введём дополнительные термины для удобства дальнейшего описания.

- **Мера** — это поле, над которым проводится агрегация (то есть поле, которое используется в качестве аргумента агрегирующей функции).
- **Измерение** — это поле, для каждого уникального значения которого проводится агрегация над мерой.

ПРИМЕР

Например, рассмотрим таблицу **payments**.

id_client	date	payment
111	10.08.2024	1500
112	12.09.2024	2250
112	10.10.2024	1000
111	15.10.2024	3700

Можно применить агрегирующую функцию **SUM** к полю **payment** по всей таблице без группировки по какому-либо полю. Результатом будет одно значение.

```
SELECT SUM(payment) AS sum_p
FROM payments
```

sum_p

8450

Теперь используем агрегирующую функцию **SUM** в комбинации с конструкцией **GROUP BY**.

```
SELECT
    id_client
    , SUM(payment) AS sum_p
FROM payments
GROUP BY id_client
```

id_client	sum_p
111	5200
112	3250

Важно:

- в качестве меры выступает поле **payment**, в качестве измерения — поле **id_client**;
- агрегирующих функций может быть несколько, как и измерений. Все измерения должны быть прописаны в **GROUP BY**;
- при применении группировки конфигурация таблицы меняется: количество строк меняется с изначального **N** на количество уникальных значений признака-измерения (или признаков-измерений).

Оконная агрегирующая функция

Оконная агрегирующая функция позволяет проводить агрегации, не меняя структуру изначальной таблицы. Группировка при этом не применяется, следовательно, количество строк не уменьшается, но добавляются дополнительные столбцы, содержащие агрегированное значение по заданному окну.

Синтаксис

```
SELECT
    АГРЕГАТНАЯ_ФУНКЦИЯ(выражение) OVER (
        [PARTITION BY столбец1, столбец2, ...]
        [ORDER BY столбец1 [ASC|DESC], столбец2 [ASC|DESC], ...]
        [ROWS | RANGE BETWEEN ...]
    )
FROM table;
```

Важно:

- **PARTITION BY** разделяет данные на группы (окна), для которых вычисляется функция;
- **ORDER BY** определяет порядок строк внутри каждой группы;
- **ROWS/RANGE BETWEEN** задаёт рамки для вычисления оконной функции.

ПРИМЕР 1

Найти сумму платежей по всей таблице.

```
SELECT
    p.*
    , SUM(payment) OVER () AS sum_p
FROM payments p
```

Результат

id_client	date	payment	sum_p
111	10.08.2024	1500	8450
112	12.09.2024	2250	8450
112	10.10.2024	1000	8450
111	15.10.2024	3700	8450

Как мы видим, оконная агрегация создаёт дополнительный столбец с результатом агрегации.

Можно было получить такой же результат с помощью подзапроса, вычислив сумму отдельным шагом и присоединив к каждой строке.

```
SELECT
    p.*
    , sum_p
FROM payments p
    JOIN (
        SELECT SUM(payment) AS sum_p
        FROM payments
    ) t
    on 1=1
```

PARTITION BY

ПРИМЕР 2

Найти сумму платежей по каждому клиенту.

-с группировкой:

```
SELECT
    id_client
    , SUM(payment) AS sum_p
FROM payments
GROUP BY id_client
```

-с оконной функцией:

```
SELECT
    *
    , SUM(payment) OVER (PARTITION BY id_client) AS sum_p
FROM payments
```

Результат 1

id_client	date
111	10.08.2024
112	12.09.2024
112	10.10.2024
111	15.10.2024

Результат 2

id_client	date	payment	sum_p
111	10.08.2024	1500	1500
112	12.09.2024	2250	3750
112	10.10.2024	1000	4750
111	15.10.2024	3700	8450

Важно:

- в первом случае мы применяем обычную группировку и получаем один результат для каждого уникального измерения, в данном случае — **id_client**;
- во втором случае количество строк не изменяется. С помощью **PARTITION BY** задаётся окно, в рамках которого сумма рассчитывается для каждого уникального значения **id_client** и записывается в новый столбец;
- применять группировку одновременно с оконной функцией невозможно.

Аналогичный результат можно было получить с помощью следующего подзапроса.

```
SELECT
    p.*
    , sum_p
FROM payments p
JOIN (
    SELECT
        id_client
        , SUM(payment) AS sum_p
    FROM payments
    GROUP BY id_client
) t
ON p.id_client = t.id_client
```

ORDER BY

Ранжировочный признак в оконной функции — это набор столбцов или выражений, которые указываются в секции **ORDER BY** внутри окна (**OVER**). Этот признак определяет в рамках каждого окна порядок строк, по которому будет рассчитываться **ранг** или любая другая оконная функция, зависящая от порядка строк.

Ранжировочный признак имеет функцию накопления. Таким образом, будет производиться кумулятивный подсчёт/сумма/среднее поля-меры в соответствии с последовательностью, задаваемой ранжировочным полем.

ПРИМЕР 1

Найти кумулятивную сумму платежей по всем клиентам.

```
SELECT
    *,
    SUM(payment) OVER (ORDER BY date) AS sum_p
FROM payments
```

Результат 1

id_client	date	payment	sum_p
111	10.08.2024	1500	1500
112	12.09.2024	2250	3750
112	10.10.2024	1000	4750
111	15.10.2024	3700	8450

Важно:

- колонка **sum_p** показывает кумулятивную сумму по всем платежам таблицы в соответствии с ранжировкой по полю даты платежа;
- первое наблюдение соответствует первому значению поля **payment**;
- последнее наблюдение соответствует сумме всех значений поля **payment**.



Дополнительное задание: как можно воспроизвести данный результат, не используя оконную агрегацию?

ПРИМЕР 2

Найти кумулятивную сумму платежей по каждому клиенту отдельно.

```
SELECT
    *,
    SUM(payment) OVER (PARTITION BY id_client ORDER BY date) AS sum_p
FROM payments
```

Результат 2

id_client	date	payment	sum_p
111	10.08.2024	1500	1500
112	12.09.2024	2250	2250
112	10.10.2024	1000	3250
111	15.10.2024	3700	5200

В этом случае мы задали и окно, и ранжировочный признак, благодаря чему накопление происходит не в рамках всей таблицы, а в рамках каждого конкретного клиента.

LAG и LEAD: оконные функции смещения

Ещё одной частой задачей бывает «подтягивание» прошлой или следующей строки (и, соответственно, значения какого-то поля этой строки). Для этого используются функции **LAG** (для **прошлого** значения) и **LEAD** (для **следующего** значения).

Синтаксис

```
SELECT
    LAG(поле-аргумент, [кол-во шагов]) OVER (
        PARTITION BY поле-окно ORDER BY ранжировочное поле
    ) AS lag_value,
    LEAD(поле-аргумент, [кол-во шагов]) OVER (
        PARTITION BY поле-окно ORDER BY ранжировочное поле
    ) AS lead_value
FROM table;
```

В качестве **поля-аргумента** функции **LAG** и **LEAD** берут на вход то поле, прошлое или следующее значение которого необходимо подтянуть.

Аргумент «кол-во шагов», заключённый в квадратные скобки, является необязательным. По умолчанию равен единице.

Оконная часть может быть опущена. В таком случае прошлая строка будет подыскиваться в рамках всей таблицы.

Ранжировочная часть определяет, по какому полю искать предыдущее значение.

По аналогии с **ROW_NUMBER** её можно (однако не рекомендуется) опустить: в таком случае ранжирование произойдёт с точки зрения последовательности хранения строк в базе данных, что не всегда может быть корректно.

Допустим, у нас есть таблица с платежами **pays**.

id_order	id_client	date_pay
1010001	101	2024-08-10
1010002	102	2024-09-17
1010003	101	2024-09-30
1010004	102	2024-10-11
1010005	101	2024-12-13

ПРИМЕР 1

Определить, сколько в среднем проходило времени между заказами.

1. Для каждой строки подтянем прошлое время `date_pay` и найдём разницу между `date_pay` и прошлым значением `date_pay`.

```
SELECT
    t.*
    , LAG(date_pay) OVER (ORDER BY date_pay) AS date_pay_lag
    , date_pay - LAG(date_pay) OVER (ORDER BY date_pay) AS delta
FROM pays
```

Результат

id_order	id_client	date_pay	date_pay_lag	delta
1010001	101	2024-08-10	NaN	NaN
1010002	102	2024-09-17	2024-08-10	37
1010003	101	2024-09-30	2024-09-17	13
1010004	102	2024-10-11	2024-09-30	11
1010005	101	2024-12-13	2024-10-11	62

Обрати внимание: в данном случае мы получим одно пустое значение, для которого нет предыдущей записи. При расчётах важно помнить об этом, так как данные потери могут быть значимыми на маленьких датасетах.

2. Теперь рассчитаем среднее время между заказами (поле `delta`).

```
SELECT AVG(delta) AS avg_delta
FROM (
    SELECT
        t.*
        , LAG(date_pay) OVER (ORDER BY date_pay) AS date_pay_lag
        , date_pay - LAG(date_pay) OVER (ORDER BY date_pay) AS delta
    FROM pays
) t
```

Результат

avg_delta

30.75

Обрати внимание: пустое значение не будет учитываться в расчёте среднего.

ПРИМЕР 2

Определить, сколько в среднем проходило времени между заказами для каждого пользователя.

Для этого:

- 1) для каждой строки подтянем прошлое время заказа с полем-окном `id_client`;
- 2) найдём разницу между `date_pay` и прошлым значением `date_pay`;
- 3) рассчитаем среднее время между заказами с группировкой по полю `id_client`.

```
SELECT
    id_client
  , AVG(date_pay - date_pay_lag) AS avg_delta
FROM (
    SELECT
        t.*
      , LAG(date_pay) OVER (
            PARTITION BY id_client ORDER BY date_pay
        ) AS date_pay_lag
      , date_pay - LAG(date_pay) OVER (
            PARTITION BY id_client ORDER BY date_pay
        ) AS delta
    FROM pays
  ) s
GROUP BY id_client
```

В данном случае прошлый заказ будет определяться не в рамках всей таблицы, а в рамках каждого уникального клиента.

Так будет выглядеть таблица-подзапрос s.

id_order	id_client	date_pay	date_pay_lag	delta
1010001	101	2024-08-10	NaN	NaN
1010002	102	2024-09-17	NaN	NaN
1010003	101	2024-09-30	2024-08-10	11
1010004	102	2024-10-11	2024-09-17	24
1010005	101	2024-12-13	2024-09-30	73

В данном случае мы ищем данные в рамках окна с одинаковым **id_client** и определяем предыдущую строку, соответственно, получим столько пустых значений, сколько имеем уникальных **id_client**.

КОЛИЧЕСТВО ШАГОВ В ФУНКЦИЯХ LAG И LEAD

В качестве второго (необязательного) аргумента функций **LAG** и **LEAD** можно указать, на сколько шагов «прошлое» или «будущее» значение необходимо подтянуть.

ПРИМЕР

Для каждого уникального клиента и каждого заказа найти позапрошлый заказ и вывести его дату, а также дельту в количестве дней.

```
SELECT
    AVG(date_pay - date_pay_lag) AS avg_delta
    , id_client
FROM (
    SELECT
        t.*
        , LAG(date_pay, 2) OVER (
            PARTITION BY id_client ORDER BY date_pay
        ) AS date_pay_lag
    FROM pays
) s
GROUP BY id_client
```

Результат

id_order	id_client	date_pay	date_pay_lag	delta
1010001	101	2024-08-10	NaN	NaN
1010002	102	2024-09-17	NaN	NaN
1010003	101	2024-09-30	NaN	NaN
1010004	102	2024-10-11	NaN	NaN
1010005	101	2024-12-13	2024-08-10	94

В данном случае мы ищем данные в рамках окна с одинаковым **id_client** и задаём шаг в две строки назад. В результате получим позапрошлый заказ только для клиента 101 и только для заказа 1010005, а все остальные строки останутся незаполненными.

Ранжирующие функции

Одной из наиболее частых задач является **проставление ранга** в соответствии с какой-то ранжировочной логикой. Рассмотрим подробнее функции, которые решают эту задачу.

ROW_NUMBER

Функция **ROW_NUMBER()** используется для проставления порядкового номера для каждой строки.

Синтаксис

```
SELECT
    ROW_NUMBER() OVER (PARTITION BY поле-окно ORDER BY
ранжировочное поле)
FROM table;
```

Важно:

- **ROW_NUMBER()** не берёт на вход аргументы, так как задача функции — проставить ранг от 1 до N и для этого не требуется проводить вычисления над какими-либо полями таблицы;
- **оконная часть PARTITION BY** может быть опущена, в таком случае проставление рангов будет проведено для всех строк таблицы совместно;
- **ранжировочная часть** указывает, на основании какого поля должны проставляться ранги. Если её опустить, ранжирование проведётся в соответствии с последовательностью хранения строк в базе данных, что не всегда может быть корректно.

Допустим, у нас есть исходная таблица с заказами **orders**.

id_client	amount_pay	city
111	150	Москва
112	2500	Казань
113	1600	Москва
114	50	Казань
115	3900	Казань
116	9500	Москва

ПРИМЕР 1

Найти порядковый номер заказа по возрастанию суммы заказа.

```
SELECT
    *,
    ROW_NUMBER() OVER (ORDER BY amount_pay ASC) AS rn
FROM orders
```

Результат

id_client	amount_pay	city	rn
111	150	Москва	2
112	2500	Казань	4
113	1600	Москва	3
114	50	Казань	1
115	3900	Казань	5
116	9500	Москва	6

ПРИМЕР 2

Найти порядковый номер заказа по возрастанию суммы заказа отдельно в рамках каждого города.

```
SELECT
    *, ROW_NUMBER() OVER (PARTITION BY city ORDER BY amount_pay ASC) AS rn
FROM orders
```

Результат

id_client	amount_pay	city	rn
111	150	Москва	2
112	2500	Казань	4
113	1600	Москва	3
114	50	Казань	1
115	3900	Казань	5
116	9500	Москва	6

Важно:

- в первом примере мы провели ранжирование без указания окна, то есть по всем данным таблицы. Во втором примере — в рамках поля **city**, то есть создали не одну систему рангов, а столько, сколько уникальных значений поля присутствует в таблице;
- чтобы изменить логику возрастания на логику убывания, необходимо приписать **DESC** после ранжировочных признаков (по аналогии с конструкцией **ORDER BY**);
- если в качестве поля-окна поставить ключ (или другое поле, уникальное для каждой строки), то все значения рангов будут равны единице;
- в большинстве диалектов (но не во всех) оконная функция не может использоваться в блоке **WHERE** или сочетаться с оператором **GROUP BY**. Таким образом, следующие примеры не смогут отработать.


```

SELECT
    *
    , ROW_NUMBER() OVER (PARTITION BY city ORDER BY amount_pay) AS rn
FROM orders
WHERE rn = 1

```

```

SELECT
    *
    , ROW_NUMBER() OVER (PARTITION BY city ORDER BY amount_pay) AS rn
FROM orders
WHERE ROW_NUMBER() OVER (PARTITION BY city ORDER BY amount_pay) = 1

```

```

SELECT
    COUNT(*) AS cnt
    , ROW_NUMBER() OVER (PARTITION BY city ORDER BY amount_pay) AS rn
FROM orders
GROUP BY rn

```

```

SELECT
    COUNT(*) AS cnt
    , ROW_NUMBER() OVER (PARTITION BY city ORDER BY amount_pay) AS rn
FROM orders
GROUP BY ROW_NUMBER() OVER (PARTITION BY city ORDER BY amount_pay)

```

- Чтобы провести фильтрацию по значению rn, необходимо построить подзапрос.

```

SELECT *
FROM (
    SELECT
        *
        , ROW_NUMBER() OVER (PARTITION BY city ORDER BY amount_pay) AS rn
    FROM orders
) t
WHERE rn = 1

```

RANK И DENSE_RANK

Иногда признак ранжирования может иметь одинаковое значение для двух или более строк. Чтобы дифференцировать такие варианты, существуют другие оконные функции ранжирования, такие как **RANK()** и **DENSE_RANK()**.

Синтаксис

SELECT

RANK() **OVER** (**PARTITION BY** поле-окно **ORDER BY** ранжировочное поле) ,

DENSE_RANK() **OVER** (**PARTITION BY** поле-окно **ORDER BY** ранжировочное поле)

FROM table;

Важно:

- оконная часть **PARTITION BY** может быть опущена, в таком случае проставление рангов будет проведено для всех строк таблицы совместно;
- при использовании **ROW_NUMBER** ранг не повторяется. Если значения ранжировочного признака (признаков) совпадают, то различные ранги проставятся этим строкам в соответствии с их логикой хранения в таблице. Таким образом, при отсутствии окна (то есть окно — это вся таблица) наибольшее значение ранга будет **равно** количеству строк в таблице;
- при использовании **DENSE_RANK** ранг повторяется для совпадающих значений ранжировочного признака и затем переходит на следующее значение ранга. Таким образом, при отсутствии окна наибольшее значение ранга будет **не равно** количеству строк в таблице;
- при использовании **RANK** ранг повторяется для совпадающих значений ранжировочного признака и затем переходит на то значение ранга, которое было бы в случае **ROW_NUMBER**, то есть «догоняет» ранги. Таким образом, при отсутствии окна наибольшее значение ранга будет **равно** количеству строк в таблице.

Допустим, у нас есть исходная таблица с возрастом клиентов, которая называется **age**.

name	age
Вася	12
Петя	13
Маша	14
Ксюша	14
Вика	15
Дима	17

ПРИМЕР

Сравнить работу функций ранжирования, используя возраст клиентов.

```
SELECT
    *
    , ROW_NUMBER() OVER (ORDER BY age) AS row_num
    , DENSE_RANK() OVER (ORDER BY age) AS dense_rnk
    , RANK() OVER (ORDER BY age) AS rnk
FROM age
```

Результат

name	age	row_num	dense_rnk	rnk
Вася	12	1	1	1
Петя	13	2	2	2
Маша	14	3	3	3
Ксюша	14	4	3	3
Вика	15	5	4	5
Дима	17	6	5	6

Как мы видим, функция **ROW_NUMBER** пронумеровала строки по порядку, а **DENSE_RANK** и **RANK** присвоили одинаковые значения строкам с одинаковым возрастом. При этом **DENSE_RANK** использовал «плотный» ранг, а **RANK** — «неплотный», то есть пропустил значения.

Ручная настройка границ окна

Когда мы используем оконные функции в SQL, чаще всего ограничиваемся простыми случаями:

- «Посчитай сумму по группе» (**PARTITION BY**);
- «Посчитай накопительный итог» (**ORDER BY**).

Но бывают задачи, когда нужно смотреть не на всё «окно до текущей строки», а только на **кусочек** этого окна: например, 3 последних заказа клиента, 7 дней вокруг даты или 5 предыдущих транзакций.

Именно для этого в SQL есть ручная настройка границ окна — **ROWS** и **RANGE BETWEEN**.

У нас есть возможность контролировать, какое количество строк будет браться для расчёта, с помощью следующих операторов:

- **ROWS BETWEEN** — оператора, который ставится после **ORDER BY** и указывает, что дальше будет правило, по которому выбираются строки окна;
- **[N] PRECEDING** — сколько строк взять до текущей (с точки зрения ранжировки), где N — целое число;
- **[N] FOLLOWING** — сколько строк взять после текущей (с точки зрения ранжировки), где N — целое число;
- **CURRENT ROW** — оператора текущей строки;
- **UNBOUNDED** — оператора неограниченного количества строк относительно текущей.

Для примера рассмотрим таблицу **dynamic_metrics**, где представлен CAC (Customer Acquisition Cost), то есть издержки на привлечение клиентов. Мы считаем, что в этих данных слишком большой разброс от дня к дню, поэтому хотим воспользоваться скользящим средним. Такая операция позволит сгладить график и при этом сохранить основной тренд в данных.

date_obs	CAC
2024-11-01	113
2024-11-02	123
2024-11-03	189
2024-11-04	86
2024-11-05	101
2024-11-06	201

Например, среднее по предыдущему, следующему и текущему значению.

ПРИМЕР 1

Заменить каждое значение CAC на среднее, посчитанное по предыдущему, текущему и следующему значениям.

```
SELECT
    *,
    AVG(CAC) OVER (ORDER BY date_obs ROWS BETWEEN 1 PRECEDING
    AND 1 FOLLOWING) AS avg_cac
FROM dynamic_metrics
```

Результат 1

date_obs	CAC	avg_cac
2024-11-01	113	NULL
2024-11-02	123	142
2024-11-03	189	133
2024-11-04	86	125
2024-11-05	101	129
2024-11-06	201	NULL

Важно:

- количество строк в таблице не изменилось, добавился столбец с агрегированной оконной функцией;
- для первой и последней строки значение не будет посчитано, так как у первого наблюдения нет предыдущего, а у последнего — следующего. Таким образом, в форме **ROWS BETWEEN M PRECEDING AND N FOLLOWING** мы будем терять **M+N** наблюдений.

Вопрос: в расчёте скольких средних участвует значение CAC 189?

Ответ: 3

ПРИМЕР 2

Заменить каждое значение CAC на среднее, посчитанное только по предыдущему и текущему значению.

```
SELECT
    *
    , AVG(CAC) OVER (
        ORDER BY date_obs
        ROWS BETWEEN 1 PRECEDING AND CURRENT ROW
    ) AS avg_cac
FROM dynamic_metrics
```

Результат 2

date_obs	CAC	avg_cac
2024-11-01	113	NULL
2024-11-02	123	118
2024-11-03	189	156
2024-11-04	86	138
2024-11-05	101	94
2024-11-06	201	151

ПРИМЕР 3

Заменить каждое значение CAC на среднее, посчитанное по текущему и следующему значению.

```
SELECT
    *,
    AVG(CAC) OVER (
        ORDER BY date_obs
        ROWS BETWEEN CURRENT ROW AND 1 FOLLOWING
    ) AS avg_cac
FROM dynamic_metrics
```

Результат 3

date_obs	CAC	avg_cac
2024-11-01	113	118
2024-11-02	123	156
2024-11-03	189	138
2024-11-04	86	94
2024-11-05	101	151
2024-11-06	201	NULL

UNBOUNDED

Оператор **UNBOUNDED** используется для того, чтобы взять в вычисление неограниченные (то есть все имеющиеся) строки слева или справа.

ПРИМЕР 1

Для каждой строки найти средний CAC за весь прошедший период.

```
SELECT
    *,
    AVG(CAC) OVER (
        ORDER BY date_obs
        ROWS BETWEEN UNBOUNDED PRECEDING AND CURRENT ROW
    ) AS avg_cac
FROM dynamic_metrics
```

Результат

date_obs	CAC	avg_cac
2024-11-01	113	113
2024-11-02	123	118
2024-11-03	189	142
2024-11-04	86	128
2024-11-05	101	122
2024-11-06	201	136

В данном запросе в расчёт среднего будут браться все строки меньше текущей с точки зрения ранжировки.

ПРИМЕР 2

Для каждой строки найти средний CAC за весь последующий период.

```
SELECT
    *
    , AVG(CAC) OVER (
        ORDER BY date_obs
        ROWS BETWEEN CURRENT ROW AND UNBOUNDED FOLLOWING
    ) AS avg_cac
FROM dynamic_metrics
```

Результат

date_obs	CAC	avg_cac
2024-11-01	113	136
2024-11-02	123	140
2024-11-03	189	144
2024-11-04	86	129
2024-11-05	101	151
2024-11-06	201	201

В данном запросе в расчёт среднего будут браться все строки **больше** текущей с точки зрения ранжировки. Операция напоминает операцию закрепления ячеек в Excel.

Несложно заметить, что два следующих запроса эквивалентны.

```
----1

SELECT
    *
    , AVG(CAC) OVER (
        ORDER BY date_obs
        ROWS BETWEEN UNBOUNDED PRECEDING AND UNBOUNDED FOLLOWING
    ) AS avg_cac
FROM dynamic_metrics

-----2

SELECT
    *
    , AVG(CAC) OVER (ORDER BY date_obs) AS avg_cac
FROM dynamic_metrics
```

Заключение

- Познакомились с понятием оконной функции.
- Узнали про агрегирующие оконные функции и их отличие от стандартной агрегации.
- Научились пользоваться функциями смещения **LAG/LEAD**.
- Познакомились с ранжирующими оконными функциями.
- Познакомились с операторами **PRECEDING**, **FOLLOWING** и **UNBOUNDED**, которые накладывают ограничения на окна.